Kubernetes administration and operations

Run clusters, not only apps: control plane and etcd, backups, safe upgrades, RBAC, node maintenance, quotas and autoscaling, network and pod security, and debugging cluster faults.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The control plane in depth (13 min)
2. etcd backup and restore (13 min)
3. Upgrading clusters safely (13 min)
4. RBAC and service accounts (13 min)
5. Nodes: draining, taints and capacity (12 min)
6. Resource quotas, limits and autoscaling (13 min)
7. Network policies and pod security (13 min)
8. Debugging cluster and node problems (14 min)
9. Monitoring the control plane and setting SLOs (12 min)
10. Persistent storage and CSI operations (13 min)
11. Multi-tenancy, namespaces and fair sharing (12 min)
12. A cluster disaster recovery runbook (13 min)
Cheat sheet

Lesson 1. The control plane in depth

13 minutes. Know what each control plane component does, how a request flows through it, and which failures matter most.

Using Kubernetes means writing manifests. Running it means keeping the control plane healthy. When you operate clusters you need to know exactly what the API server, etcd, scheduler and controller manager do, how they depend on each other, and what breaks when each one fails. This knowledge is what lets you stay calm when `kubectl` stops answering.

What you will learn

- Trace a request through the API server
- Explain what etcd stores and why it matters most
- Describe the scheduler and controller manager roles
- Predict the effect of each component failing

The API server

Every read and write goes through the **API server**. It authenticates the caller, authorises the request (RBAC), runs admission controllers that can mutate or reject objects, validates the object, and stores it in etcd. Nothing else writes to etcd. Kubelets, controllers and users are all clients of this one gateway. It is stateless and can run as several replicas behind a load balancer.

etcd

etcd is a consistent key-value store holding all cluster state: every object you create. It uses a consensus protocol and needs a majority of members to work, so three or five members are typical. It is sensitive to slow disks and network latency: slow writes show up as slow API calls. If etcd is lost with no backup, the cluster state is lost, although running containers continue for a while.

Scheduler and controllers

The **scheduler** watches for pods with no node and picks one by filtering (resources, taints, affinity) and scoring. The **controller manager** runs control loops: the Deployment controller creates ReplicaSets, the ReplicaSet controller creates pods, the node controller marks unreachable nodes, and many more. Each loop compares desired with actual state and acts to close the gap. The cloud controller manager does the same for cloud load balancers and routes.

On each node

The **kubelet** receives pod specs for its node, asks the container runtime to run them, runs probes and reports status. **kube-proxy** (or an alternative network layer) implements service routing. A node can keep running existing pods if the control plane is down, but nothing new is scheduled and nothing is repaired.

What breaks when

API server down: no changes and no `kubectl`; existing pods keep running. etcd down or slow: the API server fails or lags. Scheduler down: new pods stay Pending. Controller manager down: no scaling, no replacement of failed pods. Kubelet down on a node: that node goes NotReady and its pods are rescheduled after a timeout.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How The control plane in depth fits together

Part	What it does	What goes wrong
The API server	Every read and write goes through the API server.	Running etcd on slow shared disks.
etcd	etcd is a consistent key-value store holding all cluster state: every object you create.	No monitoring of control plane latency.
Scheduler and controllers	The scheduler watches for pods with no node and picks one by filtering (resources, taints, affinity) and scoring.	Treating the control plane as an afterthought.
On each node	The kubelet receives pod specs for its node, asks the container runtime to run them, runs probes and reports status.	Check the Common mistakes list below this diagram for what usually goes wrong here.
What breaks when	API server down: no changes and no kubectl; existing pods keep running.	Single API server in production.

Flow: The API server to etcd / etcd to Scheduler and controllers / Scheduler and controllers to On each node / On each node to What breaks when

Incident: kubectl hangs

Incident: *Engineers could not run any kubectl command and the dashboard timed out.*

```
$ kubectl get nodes
Unable to connect to the server: net/http: request canceled (Client.Timeout exceeded)
$ curl -sk https://10.0.0.10:6443/livez?verbose | tail -3
[-]etcd failed: reason withheld
livez check failed
```

The health endpoint pointed at etcd. Disk latency on the etcd members had risen after a noisy neighbour job. Moving etcd to dedicated fast disks and alerting on write latency restored the API; running workloads had stayed up the whole time.

Common mistakes

- Treating the control plane as an afterthought
- Running etcd on slow shared disks
- Single API server in production
- No monitoring of control plane latency

Check yourself

Who writes to etcd?

Answer: Only the API server

What does the scheduler do?

Answer: Assigns pending pods to nodes

What happens to running pods if the API server is down?

Answer: They keep running but nothing changes

Why use 3 or 5 etcd members?

Answer: A majority is needed to agree

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. etcd backup and restore

13 minutes. Take a verified snapshot of cluster state, restore it, and know the limits of what a backup protects.

If you lose etcd and have no backup, you lose the definition of your cluster. A backup you have never restored is only a hope. This lesson covers taking snapshots, storing them safely, restoring into a new cluster, and testing the whole procedure on a schedule.

What you will learn

- Take an etcd snapshot
- Store and protect backups
- Restore from a snapshot
- Test restores and know what a backup does not cover

Taking a snapshot

Use `etcdctl` with the endpoints and client certificates of your cluster:

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
etcdutl snapshot status /backup/etcd-2024-05-14.db -w table
```

The status command shows the revision, key count and size, which proves the file is readable. Managed Kubernetes services hide etcd; use the provider's backup features or a tool that backs up objects and volumes.

Store it well

Copy snapshots off the node at once, to storage in another location, encrypted and with limited access, because etcd holds Secrets. Keep several generations and a retention policy. Schedule snapshots often enough for your recovery point: hourly is common. Alert if the newest snapshot is older than expected.

Restoring

Restoring creates a new data directory from the snapshot with `etcdutl snapshot restore`, then the etcd static pod or service is pointed at it. On a multi-member cluster, restore every member from the same snapshot with the new cluster flags and start them together. Stop the API servers during the restore so they do not write. After starting, check member health and that objects are present.

Limits of a backup

An etcd snapshot restores Kubernetes objects. It does not contain the data in persistent volumes, container images, certificates stored outside the data directory unless you save them, or the state of external systems. It also restores old state: objects deleted since then reappear, and nodes may disagree with reality. Pair etcd backups with volume snapshots, a copy of the PKI directory, and manifests kept in a repository.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How etcd backup and restore fits together

Part	What it does	What goes wrong
Taking a snapshot	Use etcdctl with the endpoints and client certificates of your cluster.	Assuming etcd backup covers volumes.
Store it well	Copy snapshots off the node at once, to storage in another location, encrypted and with limited access, because etcd holds Secrets.	Forgetting that etcd holds Secrets.
Restoring	Restoring creates a new data directory from the snapshot with etcdutl snapshot restore, then the etcd static pod or service is pointed at it.	Never testing a restore.
Limits of a backup	An etcd snapshot restores Kubernetes objects.	Leaving snapshots on the same node.

Flow: Taking a snapshot to Store it well / Store it well to Restoring / Restoring to Limits of a backup

Incident: The backup that could not restore

> **Incident:** *A cluster lost two of three etcd members and the team tried to restore the latest nightly snapshot.*

```
$ etcdutl snapshot status /backup/etcd-latest.db -w table
Error: file is corrupt or truncated
$ ls -l /backup | tail -3
-rw-r--r-- 1 root root 0 May 13 02:00 etcd-2024-05-13.db
```

The copy job had been failing for a month and writing empty files. The team restored from an older valid file, then added a check that verifies each snapshot with the status command, alerts on size and age, and runs a quarterly restore drill in a test cluster.

Common mistakes

- Never testing a restore
- Leaving snapshots on the same node
- Forgetting that etcd holds Secrets
- Assuming etcd backup covers volumes

Check yourself

What command checks a snapshot is readable?

Answer: etcdutl snapshot status

Where should snapshots be stored?

Answer: Off the node, encrypted, access-controlled

Does an etcd snapshot include persistent volume data?

Answer: No

What should API servers do during a restore?

Answer: Be stopped so they do not write

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Upgrading clusters safely

13 minutes. Plan and run a Kubernetes version upgrade: order of components, version skew, API deprecations, nodes and rollback options.

Kubernetes ships a new minor version about three times a year, and each version is supported for roughly a year. Staying on a supported version is a security and reliability task. Upgrades are routine when you plan them and painful when they are a surprise after two years.

What you will learn

- Respect the supported upgrade path and version skew
- Find deprecated APIs before they break
- Upgrade the control plane, then nodes
- Prepare a rollback and test in a lower environment

One minor version at a time

You upgrade from 1.N to 1.N+1, not across several minors at once. The control plane is upgraded first. Kubelets may be older than the API server by a limited number of minor versions (check the version skew policy for the current allowance) but never newer. Components such as kubectl, CNI plugins, ingress controllers and operators have their own compatibility tables: read them before you start.

Deprecated APIs

Each release removes some API versions. A manifest using a removed version fails to apply. Before upgrading, scan the cluster and your repositories for deprecated APIs with tools such as `kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis` or dedicated scanners, update manifests and charts, and read the release notes for removals and behaviour changes.

Control plane then nodes

With kubeadm: upgrade the first control plane node (`kubeadm upgrade plan`, then `apply`), then the others with `kubeadm upgrade node`, then upgrade kubelet and kubectl packages and restart the kubelet. For worker nodes, upgrade one or a few at a time: **cordon** and **drain** the node, upgrade it, **uncordon**. Managed services automate much of this; you still choose the timing and check workload health. An alternative is the surge approach: add new nodes at the new version and drain the old ones.

Rollback and rehearsal

Control plane upgrades are hard to undo, so the real rollback plan is a restored etcd snapshot and a documented procedure. Take a fresh snapshot first. Rehearse in a staging cluster that matches production, and run smoke tests after each step. Use PodDisruptionBudgets so draining does not take down a service, and schedule the window when traffic is low.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs

Diagram: How Upgrading clusters safely fits together

Part	What it does	What goes wrong
One minor version at a time	You upgrade from 1.N to 1.N+1, not across several minors at once.	Skipping minor versions.
Deprecated APIs	Each release removes some API versions. A manifest using a removed version fails to apply.	Upgrading without a snapshot.
Control plane then nodes	With kubeadm: upgrade the first control plane node (kubeadm upgrade plan, then apply), then the others with kubeadm upgrade node, then upgrade kubelet and kubectl...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Rollback and rehearsal	Control plane upgrades are hard to undo, so the real rollback plan is a restored etcd snapshot and a documented procedure.	Draining without disruption budgets.

Flow: One minor version at a time to Deprecated APIs / Deprecated APIs to Control plane then nodes / Control plane then nodes to Rollback and rehearsal

Incident: Ingress objects stopped working

Incident: *After an upgrade, a team's routes disappeared and the pipeline failed to apply them.*

```
$ kubectl apply -f ingress.yaml
error: unable to recognize "ingress.yaml": no matches for kind "Ingress" in version "extensions/v1 \
    beta1"
$ kubectl api-versions | grep networking.k8s.io
networking.k8s.io/v1
```

The old API version had been removed. The team updated the manifests to the stable version, added a deprecation scan to the pipeline, and wrote the check into the upgrade checklist so the next release would show problems weeks earlier.

Common mistakes

- Skipping minor versions
- Ignoring plugin compatibility
- Upgrading without a snapshot
- Draining without disruption budgets

Check yourself

How do you upgrade across versions?

Answer: One minor version at a time

What goes first?

Answer: The control plane

What does draining do?

Answer: Evicts pods safely from a node

What is the real rollback for the control plane?

Answer: A tested etcd snapshot restore

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. RBAC and service accounts

13 minutes. Grant people and workloads only the Kubernetes API access they need, and audit who can do what.

Role-based access control decides what each identity may do with the Kubernetes API. Pods use service accounts to call that API as well. Poor RBAC is a common cause of both outages (a controller cannot do its job) and breaches (everyone is cluster-admin). The model is simple once you see its four objects.

What you will learn

- Write Roles, ClusterRoles and bindings
- Choose namespace-wide or cluster-wide scope
- Use service accounts for workloads
- Audit permissions with kubectl auth can-i

Four objects

A **Role** lists allowed verbs on resources inside one namespace. A **ClusterRole** does the same cluster-wide or for non-namespaced resources like nodes. A **RoleBinding** gives a Role (or ClusterRole) to subjects in one namespace. A **ClusterRoleBinding** gives a ClusterRole to subjects everywhere. RBAC only adds permissions; there is no deny.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: shop
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```

Subjects

Subjects are users, groups and service accounts. Kubernetes does not store users: they come from your authentication method (certificates, OpenID Connect through a company identity provider). Bind groups rather than individuals so access follows team membership.

Service accounts

Each pod runs as a **service account**, whose short-lived token is mounted into the pod so the application can call the API. The default service account in a namespace should have no extra rights, and pods that never call the API should not mount a token (`automountServiceAccountToken: false`). Give each application its own service account with only the verbs it needs.

Avoid broad rights

Be careful with wildcards (`*`), the `cluster-admin` role, and powerful verbs such as `create` on pods, `escalate`, `bind`, `impersonate`, and reading Secrets cluster-wide: each can be used to gain more access. Review ClusterRoleBindings regularly.

Checking access

Ask the cluster: `kubectl auth can-i delete pods -n shop --as=jane` and `kubectl auth can-i --list -n shop --as=system:serviceaccount:shop:web`. These answer questions without trial and error.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How RBAC and service accounts fits together

Part	What it does	What goes wrong
Four objects	A Role lists allowed verbs on resources inside one namespace.	Binding cluster-admin to fix errors.
Subjects	Subjects are users, groups and service accounts.	Using the default service account for everything.
Service accounts	Each pod runs as a service account, whose short-lived token is mounted into the pod so the application can call the API.	Never reviewing bindings.
Avoid broad rights	Be careful with wildcards (*), the cluster-admin role, and powerful verbs such as create on pods, escalate, bind, impersonate, and reading Secrets cluster-wide: each can...	Granting wildcards.
Checking access	Ask the cluster: kubectl auth can-i delete pods -n shop --as=jane and kubectl auth can-i --list -n shop --as=system:serviceaccount:shop:web.	These answer questions without trial and error.

Flow: Four objects to Subjects / Subjects to Service accounts / Service accounts to Avoid broad rights / Avoid broad rights to Checking access

Incident: Forbidden for the controller

Incident: *A custom controller crashed after deployment with permission errors.*

```
$ kubectl logs deploy/cert-sync | tail -2
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:ops:cert-sync" ca \
    nnot list resource "secrets" in API group "" in the namespace "shop"
$ kubectl auth can-i list secrets -n shop --as=system:serviceaccount:ops:cert-sync
no
```

The error named the exact subject, verb and resource. The team added a Role for that one verb in that one namespace and a binding, and did not use cluster-admin. They then checked the other namespaces with can-i --list.

Common mistakes

- Binding cluster-admin to fix errors
- Using the default service account for everything
- Granting wildcards
- Never reviewing bindings

Check yourself

Does RBAC support deny rules?

Answer: No, it only grants

What binds a Role to a user in one namespace?

Answer: A RoleBinding

How do you test access?

Answer: kubectl auth can-i

Where should pods that never call the API get tokens?

Answer: They should not mount one

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Nodes: draining, taints and capacity

12 minutes. Maintain nodes without downtime, control placement with taints and affinity, and plan capacity.

Nodes need patching, replacing and resizing. Doing this without dropping traffic depends on understanding how Kubernetes evicts pods, respects disruption budgets and places workloads. It also helps to keep special nodes (GPUs, large memory, spot capacity) for the workloads that need them.

What you will learn

- Cordon, drain and uncordon
- Use PodDisruptionBudgets
- Control placement with taints, tolerations and affinity
- Read node capacity and allocatable resources

Cordon and drain

`kubectl cordon node-1` marks the node unschedulable but leaves pods running. `kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data` evicts pods so they are recreated elsewhere, respecting their disruption budgets. After maintenance, `kubectl uncordon node-1` makes it schedulable again. Pods that are not managed by a controller are not recreated, so drain warns about them.

PodDisruptionBudgets

A **PodDisruptionBudget** says how many replicas of an application must remain available (`minAvailable: 2`) or may be unavailable (`maxUnavailable: 1`) during voluntary disruptions like drains. A drain waits instead of violating the budget. A budget that requires all replicas makes drains hang forever, and with a single replica a strict budget blocks maintenance.

Taints and tolerations

A **taint** on a node repels pods unless they have a matching **toleration**. For example, `kubectl taint nodes gpu-1 gpu=true:NoSchedule` keeps ordinary pods away from GPU nodes; GPU workloads tolerate the taint. Effects are NoSchedule, PreferNoSchedule and NoExecute (which also evicts running pods). Kubernetes itself adds taints such as `node.kubernetes.io/not-ready`.

Affinity and spread

Node affinity attracts pods to labelled nodes. Pod anti-affinity and topology spread constraints spread replicas across nodes and zones so one failure does not remove them all. Taints keep others out; affinity pulls pods in; use both for dedicated pools.

Capacity

`kubectl describe node` shows **capacity**, **allocatable** (what pods can use after system reservations) and what is already requested. Scheduling uses requests, not usage. Leave headroom for draining a node: if the cluster cannot absorb the pods of one node, maintenance will leave pods Pending.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Nodes: draining, taints and capacity fits together

Part	What it does	What goes wrong
Cordon and drain	kubectl cordon node-1 marks the node unschedulable but leaves pods running.	Forgetting daemonsets and local data flags.
PodDisruptionBudgets	A PodDisruptionBudget says how many replicas of an application must remain available (minAvailable: 2) or may be unavailable (maxUnavailable: 1) during voluntary...	PDBs that allow zero disruptions.
Taints and tolerations	A taint on a node repels pods unless they have a matching toleration.	Using taints without tolerations.
Affinity and spread	Node affinity attracts pods to labelled nodes.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Capacity	kubectl describe node shows capacity, allocatable (what pods can use after system reservations) and what is already requested.	Draining without enough spare capacity.

Flow: Cordon and drain to PodDisruptionBudgets / PodDisruptionBudgets to Taints and tolerations / Taints and tolerations to Affinity and spread / Affinity and spread to Capacity

Incident: The drain that never finished

> **Incident:** *Node maintenance stalled for an hour with the message "Cannot evict pod as it would violate the pod's disruption budget".*

```
$ kubectl drain node-3 --ignore-daemonsets
error when evicting pods/"api-7f6c" -n shop: Cannot evict pod as it would violate the pod's disrup \
    tion budget.
$ kubectl get pdb -n shop
NAME  MIN AVAILABLE  ALLOWED DISRUPTIONS
api   3              0
```

The deployment had 3 replicas and required 3. The team scaled it to 4, changed the budget to minAvailable 2, and the drain continued. They added a review check for budgets that allow zero disruptions.

Common mistakes

- PDBs that allow zero disruptions
- Draining without enough spare capacity
- Forgetting daemonsets and local data flags
- Using taints without tolerations

Check yourself

What does cordon do?

Answer: Stops new pods from scheduling to the node

What does a PDB protect?

Answer: Availability during voluntary disruptions

What is a taint?

Answer: A node mark that repels pods without a toleration

What does the scheduler use to place pods?

Answer: Resource requests

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Resource quotas, limits and autoscaling

13 minutes. Share a cluster fairly with requests, limits, quotas and LimitRanges, and scale pods and nodes automatically.

A shared cluster needs rules about how much each team and workload can use. Without them, one runaway deployment can starve everything else. With sensible requests, limits, quotas and autoscaling, the cluster stays predictable and cost under control.

What you will learn

- Set requests and limits and know their effects
- Apply ResourceQuotas and LimitRanges per namespace
- Use the Horizontal Pod Autoscaler
- Understand node autoscaling

Requests and limits

A **request** is what the scheduler reserves; a **limit** is the ceiling. Exceeding the memory limit gets a container OOM-killed. Exceeding the CPU limit throttles it. The ratio of requests to limits sets the quality of service class: Guaranteed (equal), Burstable, or BestEffort (none), which is evicted first under pressure. Many teams set memory request equal to limit, and set CPU requests without a tight CPU limit to avoid throttling.

Quotas and defaults

A **ResourceQuota** caps the total CPU, memory, storage or object counts in a namespace. A **LimitRange** sets default requests and limits and minimums or maximums per container, so pods without values still get sensible ones. With a quota on CPU or memory, pods must declare those resources or be rejected.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a
  namespace: team-a
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 64Gi
    pods: "100"
```

Horizontal Pod Autoscaler

The HPA changes the replica count based on metrics such as CPU use compared with the request, or custom metrics. It needs the metrics server and CPU requests set. Set sensible minimum and maximum replicas, and remember it reacts over minutes: for sudden spikes keep enough baseline. Vertical autoscaling resizes requests and is useful for right-sizing, but changes may restart pods.

Node autoscaling

The cluster autoscaler or Karpenter adds nodes when pods are Pending for lack of capacity and removes under-used ones. It relies on accurate requests: if requests are inflated you pay for idle nodes, and if they are too low pods are packed too tightly and suffer. Watch for pods that cannot be moved (local storage, strict budgets) which block scale-down.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Resource quotas, limits and autoscaling fits together

Part	What it does	What goes wrong
Requests and limits	A request is what the scheduler reserves; a limit is the ceiling.	CPU limits that cause throttling.
Quotas and defaults	A ResourceQuota caps the total CPU, memory, storage or object counts in a namespace.	Quotas without defaults.
Horizontal Pod Autoscaler	The HPA changes the replica count based on metrics such as CPU use compared with the request, or custom metrics.	No requests set.
Node autoscaling	The cluster autoscaler or Karpenter adds nodes when pods are Pending for lack of capacity and removes under-used ones.	HPA without requests.

Flow: Requests and limits to Quotas and defaults / Quotas and defaults to Horizontal Pod Autoscaler / Horizontal Pod Autoscaler to Node autoscaling

Incident: Namespace full

Incident: *A deployment could not create pods and showed "exceeded quota" events.*

```
$ kubectl describe quota team-a -n team-a
Resource          Used  Hard
requests.memory   63Gi  64Gi
$ kubectl get events -n team-a | tail -1
FailedCreate  exceeded quota: team-a, requested: requests.memory=4Gi, used: 63Gi, limited: 64Gi
```

The team had grown beyond its quota. They checked actual use, found three over-sized deployments, reduced their requests, and agreed a higher quota through a review rather than deleting the limit.

Common mistakes

- No requests set
- CPU limits that cause throttling
- Quotas without defaults
- HPA without requests

Check yourself

What does the scheduler use?

Answer: Requests

What happens when a container exceeds its memory limit?

Answer: It is OOM-killed

What does a LimitRange provide?

Answer: Default and bounds for containers

What does the HPA need to scale on CPU?

Answer: CPU requests and a metrics source

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Network policies and pod security

13 minutes. Restrict pod-to-pod traffic with network policies and restrict what containers may do with Pod Security Standards.

By default every pod can talk to every other pod, and a container can ask for powers it should not have. Two features close those gaps: NetworkPolicies limit traffic, and Pod Security Admission limits what pods can request. Together they contain damage when something is compromised.

What you will learn

- Write a default-deny NetworkPolicy and add allowances
- Know that a policy needs a network plugin that enforces it
- Apply Pod Security Standards by namespace
- Harden containers with a security context

Default deny, then allow

A NetworkPolicy selects pods and lists allowed ingress and egress. Once any policy selects a pod for a direction, only listed traffic is allowed in that direction.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
```

Then add narrow policies, for instance letting the web pods reach the API pods on one port, and allowing DNS egress. Remember DNS or everything breaks.

Enforcement

Policies are enforced by the network plugin (such as Calico or Cilium). If your plugin does not support them, the objects are accepted but do nothing. Test with a pod that should be blocked and confirm it is.

Pod Security Standards

Three profiles: **privileged** (no limits), **baseline** (blocks known escalations such as privileged containers and host namespaces) and **restricted** (current hardening practice: non-root, no privilege escalation, dropped capabilities, seccomp). Pod Security Admission applies them per namespace with labels in modes enforce, audit and warn.

```
kubectl label namespace shop pod-security.kubernetes.io/enforce=baseline pod-security.kubernetes.i \
    o/warn=restricted
```

Security context

Set it on pods and containers: `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true`, `capabilities: { drop: ["ALL"] }` and a seccomp profile. Avoid `hostNetwork`, `hostPID`, `hostPath` and privileged mode. Add policy engines such as Kyverno or OPA Gatekeeper for rules beyond the built-in profiles.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Network policies and pod security fits together

Part	What it does	What goes wrong
Default deny, then allow	A NetworkPolicy selects pods and lists allowed ingress and egress.	Assuming policies work without a supporting plugin.
Enforcement	Policies are enforced by the network plugin (such as Calico or Cilium).	Check the Common mistakes list below this diagram for what usually goes wrong here.
Pod Security Standards	Three profiles: privileged (no limits), baseline (blocks known escalations such as privileged containers and host namespaces) and restricted (current hardening practice...	Running containers as root.
Security context	Set it on pods and containers: runAsNonRoot: true, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: { drop: ["ALL"] } and a seccomp profile.	Forgetting DNS in egress rules.

Flow: Default deny, then allow to Enforcement / Enforcement to Pod Security Standards / Pod Security Standards to Security context

Incident: Lateral movement

Incident: *A compromised front-end pod scanned the cluster and reached the database pods directly.*

```
$ kubectl exec web-5d -- nc -zv db.shop.svc 5432
db.shop.svc (10.96.14.7:5432) open
$ kubectl get networkpolicy -n shop
No resources found in shop namespace.
```

There were no policies, so every pod could reach the database. The team added default deny, allowed only the API pods to reach the database on its port, enabled the restricted profile for new namespaces, and tested that web pods are now blocked.

Common mistakes

- Assuming policies work without a supporting plugin
- Forgetting DNS in egress rules
- Running containers as root
- Privileged pods for convenience

Check yourself

What is the effect of a pod being selected by an ingress policy?

Answer: Only listed ingress is allowed to it

What enforces NetworkPolicies?

Answer: The network plugin

Which profile is strictest?

Answer: Restricted

Which setting prevents gaining more privileges?

Answer: allowPrivilegeEscalation: false

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Debugging cluster and node problems

14 minutes. Find the cause of NotReady nodes, Pending pods, evictions and a slow API server using an ordered checklist.

Cluster problems feel dramatic, but they respond to an ordered method: confirm the symptom, find the layer (workload, node, control plane, network), gather evidence, change one thing, and verify. This lesson gives you the checks and commands for the failures you will meet most.

What you will learn

- Triage from workload to node to control plane
- Diagnose a NotReady node
- Diagnose Pending pods and evictions
- Check the control plane and API server health

Order of checks

Start at the symptom. A pod problem: `kubectl describe pod` (events), `kubectl logs --previous`. If the pod is fine, look at the node: `kubectl get nodes`, `kubectl describe node`. If several nodes or the API misbehave, look at the control plane. Resist changing things before you know which layer is at fault.

NotReady nodes

`kubectl describe node` shows conditions: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable. On the node, check `systemctl status kubelet`, `journalctl -u kubelet -n 100`, the container runtime (`crictl ps`, `systemctl status containerd`), disk space and inodes (`df -h`, `df -i`), memory, and the network plugin pods. Common causes: full disk, a stopped kubelet or runtime, expired certificates, or lost network.

Pending and evicted pods

A Pending pod shows the reason in events: insufficient CPU or memory, node selector or affinity mismatch, untolerated taints, unbound volume claims. Evicted pods usually indicate node pressure (often ephemeral storage or memory); look at which pods use the most and at requests. CrashLoopBackOff and ImagePullBackOff are workload-level problems, not node-level.

Control plane health

Check `kubectl get --raw=/readyz?verbose`, API server logs, etcd health and disk latency, and certificate expiry (`kubeadm certs check-expiration`). A slow API often comes from etcd latency, an overloaded API server due to a misbehaving client making huge list calls, or admission webhooks that time out and block writes.

Close the loop

Record the timeline, the cause, the fix and a prevention (an alert on disk usage or certificate expiry, a quota, a runbook). Practise these in a test cluster so the first time is not production.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Debugging cluster and node problems fits together

Part	What it does	What goes wrong
Order of checks	Start at the symptom. A pod problem: kubectl describe pod (events), kubectl logs --previous.	Restarting things before looking at events.
NotReady nodes	kubectl describe node shows conditions: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable.	Ignoring node conditions.
Pending and evicted pods	A Pending pod shows the reason in events: insufficient CPU or memory, node selector or affinity mismatch, untolerated taints, unbound volume claims.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Control plane health	Check kubectl get --raw=/readyz?verbose, API server logs, etcd health and disk latency, and certificate expiry (kubeadm certs check-expiration).	No monitoring of certificate expiry.
Close the loop	Record the timeline, the cause, the fix and a prevention (an alert on disk usage or certificate expiry, a quota, a runbook).	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Order of checks to NotReady nodes / NotReady nodes to Pending and evicted pods / Pending and evicted pods to Control plane health / Control plane health to Close the loop

Incident: Certificates expired

Incident: *All kubectl commands failed with a certificate error one morning.*

```
$ kubectl get nodes
Unable to connect to the server: x509: certificate has expired or is not yet valid
$ sudo kubeadm certs check-expiration | head -4
CERTIFICATE  EXPIRES  RESIDUAL TIME
admin.conf   Mar 01, 2025 02:11 UTC  <invalid>
```

The cluster certificates, issued a year earlier, had expired. The team renewed them with kubeadm, restarted the control plane components, then added a monitor on expiry dates and an annual upgrade schedule that renews them automatically.

Common mistakes

- Restarting things before looking at events
- Ignoring node conditions
- No monitoring of certificate expiry
- Debugging the wrong layer

Check yourself

Where do you start for a failing pod?

Answer: kubectl describe and logs

What shows node pressure?

Answer: Node conditions in kubectl describe node

Why might the API server be slow?

Answer: etcd latency, heavy clients or slow webhooks

What prevents an expiry outage?

Answer: Monitoring and renewal of certificates

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Monitoring the control plane and setting SLOs

12 minutes. Watch API server latency, errors, etcd health and controller queues, and define service levels for the cluster itself.

When the control plane is unhealthy, every workload suffers, yet many teams only watch the nodes. A small set of control plane metrics gives early warning and a target to manage against.

What you will learn

- The key API server, etcd and controller metrics
- Spotting abusive clients and expensive requests
- Certificates and other expiry risks
- Platform SLOs and alerts

Key signals

```
# API server request latency, 99th percentile (non-watch)
histogram_quantile(0.99, sum by (le, verb) (rate(apiserver_request_duration_seconds_bucket{verb!~" \
    WATCH|CONNECT"}[5m])))

# API server error ratio
sum(rate(apiserver_request_total{code=~"5.."}[5m])) / sum(rate(apiserver_request_total[5m]))

# etcd disk sync latency
histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))

# etcd database size
etcd_mvcc_db_total_size_in_bytes
```

Slow `fsync` usually means slow disks for etcd. A growing database size means too many objects or no compaction. Also watch scheduler queue depth and controller work queue depth and retries.

Find heavy clients

Large list calls without pagination, aggressive polling and controllers in restart loops can overload the API server. API Priority and Fairness provides metrics by flow.

```
kubectl get --raw /metrics | grep -E "apiserver_flowcontrol_rejected_requests_total" | head
kubectl get --raw /metrics | grep apiserver_request_total | sort -t' ' -k2 -nr | head
```

Identify the user agent or service account making the most requests, and fix the client.

Expiry

```
apiserver_client_certificate_expiration_seconds_count
min(apiserver_client_certificate_expiration_seconds_bucket)
```

Alert well before certificates expire, and rehearse renewal. Also track the API server's own serving certificate and tokens used by controllers.

SLOs

Define platform objectives such as 99.9 percent of API reads under 1 second, and pod scheduling within 30 seconds at the 95th percentile. Alert on **burn rate** and review the numbers monthly with platform users.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Monitoring the control plane and setting SLOs fits together

Part	What it does	What goes wrong
Key signals	Slow fsync usually means slow disks for etcd.	Ignoring request volume by client.
Find heavy clients	Large list calls without pagination, aggressive polling and controllers in restart loops can overload the API server.	Find heavy clients Large list calls without pagination, aggressive polling and controllers in restart loops can overload the API server.
Expiry	Alert well before certificates expire, and rehearse renewal.	Missing certificate expiry alerts.
SLOs	Define platform objectives such as 99.9 percent of API reads under 1 second, and pod scheduling within 30 seconds at the 95th percentile.	No SLO for the platform.

Flow: Key signals to Find heavy clients / Find heavy clients to Expiry / Expiry to SLOs

Incident: The controller that listed everything

Incident: *One controller's polling slowed the whole API.*

```
verb=LIST resource=pods user-agent=custom-reconciler/0.3
4800 requests/min
API p99 latency: 3.8s
```

A team's controller listed every pod in the cluster every few seconds. API latency rose for all tenants. After finding it from request metrics, the team switched it to a watch with an informer cache and added a rule to catch similar clients.

Common mistakes

- Monitoring only nodes and pods
- Ignoring request volume by client
- Missing certificate expiry alerts
- No SLO for the platform

Check yourself

What does slow etcd fsync suggest?

Answer: Slow disk

How find a heavy API client?

Answer: Request metrics by user agent or account

Why alert early on certificate expiry?

Answer: Expired certificates break the control plane

What is a platform SLO example?

Answer: API reads under 1 second 99.9 percent of the time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Persistent storage and CSI operations

13 minutes. Operate persistent volumes with storage classes and CSI drivers, handle expansion, snapshots and stuck volumes, and keep stateful workloads safe.

Kubernetes separates what an app asks for, a claim, from what the platform provides, a volume. The Container Storage Interface connects that model to real storage systems, and most operational problems come from the seams between them.

What you will learn

- StorageClass, PVC, PV and the CSI driver
- Expanding volumes and reclaim policies
- Snapshots and restore
- Debugging attach, mount and stuck volume problems

The objects

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: csi.example.com
allowVolumeExpansion: true
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: ssd
```

`WaitForFirstConsumer` delays creating the volume until a pod is scheduled, so it is created in the right zone. `reclaimPolicy: Retain` keeps the data when a claim is deleted, which is safer for important data than `Delete`.

Expand and snapshot

```
kubectl patch pvc data-db-0 -n shop -p '{"spec":{"resources":{"requests":{"storage":"200Gi"}}}}'
kubectl get pvc data-db-0 -n shop -w
```

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: db-before-upgrade
  namespace: shop
spec:
  volumeSnapshotClassName: csi-snapclass
  source:
    persistentVolumeClaimName: data-db-0
```

Volumes can usually only grow. Snapshots are not backups on their own if they live in the same storage system, so copy important data elsewhere too.

Debugging

```
kubectl describe pod db-0 -n shop | tail -20
kubectl get volumeattachment | grep db
kubectl logs -n kube-system ds/csi-node -c driver | tail
```

Typical errors are `FailedAttachVolume` (the volume is still attached to another node), `FailedMount` (a node or driver problem) and a PVC stuck in `Pending` (no matching class or no capacity). Read pod events first, then VolumeAttachment objects, then the driver logs. Never delete a VolumeAttachment or force detach without making sure the old node no longer writes to the disk.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Persistent storage and CSI operations fits together

Part	What it does	What goes wrong
The objects	WaitForFirstConsumer delays creating the volume until a pod is scheduled, so it is created in the right zone.	Reclaim policy Delete for critical data.
Expand and snapshot	Volumes can usually only grow. Snapshots are not backups on their own if they live in the same storage system, so copy important data elsewhere too.	Snapshots as the only backup.
Debugging	Typical errors are FailedAttachVolume (the volume is still attached to another node), FailedMount (a node or driver problem) and a PVC stuck in Pending (no matching...	Forcing detach without fencing the node.

Flow: The objects to Expand and snapshot / Expand and snapshot to Debugging

Incident: Multi-attach error

Incident: *A pod could not start because the volume still belonged to a failed node.*

```
Warning FailedAttachVolume: Multi-Attach error for volume "pvc-5c1f"
Volume is already exclusively attached to one node
node worker-7 is NotReady
```

The old node stopped reporting but could still be writing. The team waited for the detach timeout and confirmed the node was powered off before forcing detachment. They later added fencing and documented the procedure.

Common mistakes

- Reclaim policy Delete for critical data
- Snapshots as the only backup
- Forcing detach without fencing the node
- Expanding volumes without checking filesystem support

Check yourself

What does WaitForFirstConsumer do?

Answer: Creates the volume where the pod is scheduled

What does Retain keep?

Answer: The volume data after the claim is deleted

What does FailedAttachVolume often mean?

Answer: It is still attached elsewhere

Why add backups beyond snapshots?

Answer: Snapshots may share the same failure domain

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Multi-tenancy, namespaces and fair sharing

12 minutes. Share a cluster between teams with namespaces, quotas, network policy, RBAC and admission rules, and know when to use separate clusters.

A single large cluster is cheaper to run than many small ones, but tenants can affect each other. Good tenancy design limits resources, access and network reach per team.

What you will learn

- Namespaces as the unit of isolation
- Quotas, limit ranges and priority
- RBAC, network policy and policy controls
- When separate clusters are the better choice

A tenant namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: team-payments
  labels: {team: payments, cost-center: cc-204}
---
apiVersion: v1
kind: ResourceQuota
metadata: {name: compute, namespace: team-payments}
spec:
  hard:
    requests.cpu: "40"
    requests.memory: 160Gi
    limits.memory: 200Gi
    pods: "200"
---
apiVersion: v1
kind: LimitRange
metadata: {name: defaults, namespace: team-payments}
spec:
  limits:
    - type: Container
      default: {cpu: 500m, memory: 512Mi}
      defaultRequest: {cpu: 100m, memory: 128Mi}
```

A **ResourceQuota** caps the total a namespace may request. A **LimitRange** sets defaults and bounds per container, so pods without requests still get sensible ones.

Access and network

Use RBAC roles bound to team groups in their own namespaces. Add a default-deny **NetworkPolicy** and allow only needed flows. Enforce Pod Security Standards and admission policies, so tenants cannot run privileged pods or use host networking.

Fair scheduling

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: {name: platform-critical}
value: 100000
```

Priority classes decide who is evicted first under pressure. Reserve high priority for platform components. Use node pools or taints to separate workloads such as batch and latency-sensitive services.

MARGA by EknathaLabs
marga.eknathalabs.com

Limits of sharing

Namespaces do not isolate the kernel or control plane. If tenants are untrusted or have different compliance needs, use separate clusters or sandboxed runtimes. Track usage per team with labels and review quotas quarterly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Multi-tenancy, namespaces and fair sharing fits together

Part	What it does	What goes wrong
A tenant namespace	A ResourceQuota caps the total a namespace may request.	No default requests and limits.
Access and network	Use RBAC roles bound to team groups in their own namespaces.	Namespaces with no quotas.
Fair scheduling	Priority classes decide who is evicted first under pressure.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Limits of sharing	Namespaces do not isolate the kernel or control plane.	Assuming namespaces isolate the kernel.

Flow: A tenant namespace to Access and network / Access and network to Fair scheduling / Fair scheduling to Limits of sharing

Incident: The batch job that ate the nodes

Incident: *A team's job consumed all memory and evicted others.*

```
node memory pressure: True
Evicted: 31 pods from 6 namespaces
namespace team-batch has no ResourceQuota
```

A new batch namespace had no quota or limits. A job requested no memory and then grew until nodes were under pressure. Admission now rejects namespaces without a quota, and batch work runs on a separate node pool with lower priority.

Common mistakes

- Namespaces with no quotas
- No default requests and limits
- Assuming namespaces isolate the kernel
- Mixed trust tenants in one cluster

Check yourself

What does a ResourceQuota do?

Answer: Caps total resources in a namespace

What does a LimitRange do?

Answer: Sets defaults and bounds per container

When use separate clusters?

Answer: When isolation needs are strict

What decides who is evicted first?

Answer: Priority classes and QoS

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. A cluster disaster recovery runbook

13 minutes. Plan for losing a cluster: what to back up, how to rebuild from code and how to restore state, with tests and clear recovery targets.

Most clusters are cattle if you built them from code, but their data is not. A good disaster plan treats the two separately, gives each a target time and tests both.

What you will learn

- What lives where: code, config, secrets and data
- Rebuilding the platform from infrastructure code
- Restoring state from etcd and volume backups
- Rehearsing and timing the recovery

Classify what you have

- **Infrastructure:** networks, nodes and control plane, defined in code.
- **Cluster configuration:** add-ons, policies and applications, defined in Git.
- **Secrets and certificates:** in a secret manager and a protected backup.
- **State:** databases and volumes, covered by backups and snapshots.

Define an **RPO** (how much data loss is acceptable) and an **RTO** (how long recovery may take) for each tier of service.

Rebuild

```
terraform -chdir=infra/prod apply           # network, control plane, node pools
./bootstrap/install-delivery-agent.sh         # installs the delivery agent
kubectl apply -f bootstrap/root-app.yaml    # pulls platform add-ons and apps from Git
kubectl get nodes,pods -A | head -30
```

The order matters: infrastructure, cluster add-ons such as networking and storage drivers, secrets operator, then applications. Write the steps as an executable runbook.

Restore state

```
# etcd restore is a last resort for a self-managed control plane
ETCDCTL_API=3 etcdctl snapshot restore snapshot.db --data-dir /var/lib/etcd-restore
# volumes: restore from snapshot or backup tool
velero restore create --from-backup nightly-2026-11-04 --include-namespaces shop
```

Managed control planes do not give you etcd, so rely on application manifests in Git and a backup tool for namespaces and volumes. Databases should have their own backups and point-in-time recovery.

Rehearse

Run a game day in an empty account or region at least twice a year. Time each step, record surprises and fix the gaps. Keep the runbook and credentials available outside the cluster being recovered.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How A cluster disaster recovery runbook fits together

Part	What it does	What goes wrong
Classify what you have	Infrastructure: networks, nodes and control plane, defined in code.	Backups that were never restored.
Rebuild	The order matters: infrastructure, cluster add-ons such as networking and storage drivers, secrets operator, then applications.	No order of steps for the rebuild.
Restore state	Managed control planes do not give you etcd, so rely on application manifests in Git and a backup tool for namespaces and volumes.	Restore state Managed control planes do not give you etcd, so rely on application manifests in Git and a backup tool for namespaces and...
Rehearse	Run a game day in an empty account or region at least twice a year.	Runbooks stored only inside the failed cluster.

Flow: Classify what you have to Rebuild / Rebuild to Restore state / Restore state to Rehearse

Incident: The database nobody had restored

Incident: *A cluster rebuilt quickly but databases took a day to recover.*

```
cluster rebuilt in 2h05m
restore database: backup chain incomplete
database online after 26h
```

The platform was fully automated and rehearsed, but the databases were only assumed to be backed up. A missing archive segment lengthened the recovery. The team added monthly restore tests per database and included them in the disaster drill.

Common mistakes

- Backups that were never restored
- Runbooks stored only inside the failed cluster
- No order of steps for the rebuild
- Treating secrets as an afterthought

Check yourself

What are the two halves of cluster DR?

Answer: Rebuild from code and restore state

What does RTO mean?

Answer: How long recovery may take

Where keep the runbook?

Answer: Outside the cluster it recovers

How find gaps?

Answer: Rehearse the recovery

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Kubernetes administration and operations

etcd backup, upgrades, node maintenance, RBAC, quotas, policies and a cluster debugging checklist.

etcd snapshot

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
etcdutl snapshot status /backup/etcd.db -w table
```

Warning: *Snapshots contain Secrets. Encrypt them, copy them off the node and test a restore on a schedule.*

Upgrade with kubeadm

```
kubeadm upgrade plan
kubeadm upgrade apply v1.NN.X
kubeadm upgrade node
kubeadm certs check-expiration
```

Warning: *Upgrade one minor version at a time, control plane first, and scan for removed APIs before you begin.*

Node maintenance

```
kubectl cordon node-1
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data
kubectl uncordon node-1
kubectl taint nodes gpu-1 gpu=true:NoSchedule
```

Warning: *A PodDisruptionBudget that allows zero disruptions makes drain hang. Check budgets before maintenance.*

RBAC

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: shop
subjects:
  - kind: Group
    name: shop-developers
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Warning: *Do not bind cluster-admin to fix a permission error. Read the Forbidden message and grant the one verb on the one resource.*

Check permissions

```
kubectl auth can-i delete pods -n shop --as=jane
kubectl auth can-i --list -n shop --as=system:serviceaccount:shop:web
```

MARGA by EknathaLabs marga.eknathalabs.com

Quotas and autoscaling

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a
  namespace: team-a
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 64Gi
    pods: "100"
```

Warning: *The scheduler uses requests, not usage. Without realistic requests, autoscalers and quotas mislead you.*

Default deny network policy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
```

Warning: *After default deny, allow DNS egress and the specific flows you need, and confirm your network plugin enforces policies.*

Debugging order

```
kubectl describe pod NAME
kubectl get nodes
kubectl describe node NODE
journalctl -u kubelet -n 100
kubectl get --raw=/readyz?verbose
```

MARGA by EknathaLabs
marga.eknathalabs.com